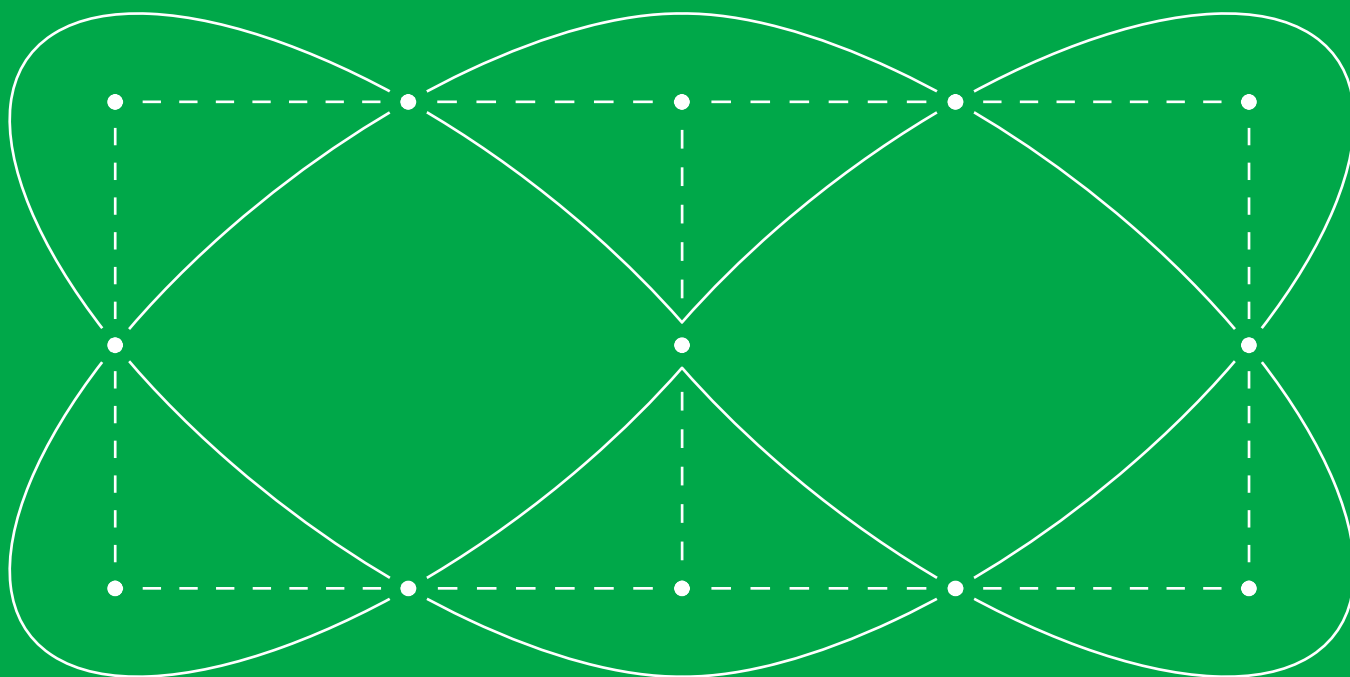


Разработка запросов к одной таблице



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 2

Введение

Контекст

При работе в команде очень важно найти общий язык со своими коллегами. Работая с данными, мы сталкиваемся с аналогичным вызовом. Одним из наиболее популярных языков программирования по работе с базами данных является SQL. Он остаётся популярным почти 50 лет после своего появления, потому что он просто работает. Базы данных были изобретены для удовлетворения потребности в хранении и использовании данных, а SQL был изобретён для взаимодействия с этими базами данных, то есть он быстро стал языком де-факто. На SQL выражаются все действия, которые можно провести с данными: от записи и чтения до администрирования сервера СУБД.

Краткий план

В этом уроке мы пройдем основы. Познакомимся с основными предложениями, которые позволяют определить:

- из какой таблицы будут выведены данные;
- какие данные будут выведены по результатам запроса;
- какие ограничения будут применены к базе данных, прежде чем из неё будут выведены данные.

Также разберём наиболее популярные встроенные функции, которые сильно упрощают жизнь при выполнении той или иной бизнесовой задачи.

Примеры рассматриваем на основе таблицы **orders**.

order_id	customer_id	merchant_id	order_amount	order_date	delivery_date	order_status	delivery_status
1	1	101	1000	2023-01-01	2023-01-03	paid	delivered
2	2	102	2000	2023-01-02	2023-01-04	paid	delivered
3	1	103	3000	2023-01-03	2023-01-06	completed	in transit
4	3	104	1500	2023-01-04	2023-01-05	completed	delivered
5	2	101	5000	2023-01-05	2023-01-07	paid	delivered

Основные предложения в SQL

Язык SQL — это в первую очередь язык запросов, который очень похож на естественный язык.

Каждый раз, когда требуется провести ту или иную операцию в БД, требуется составить корректный запрос в терминах SQL. Разберём на примере, составив следующий запрос.

ПРИМЕР

ПРОЧИТАТЬ всё ИЗ ТАБЛИЦЫ «заказы».

В терминах языка SQL запрос будет выглядеть так.

```
SELECT *  
FROM orders
```

SELECT И FROM

В приведённом выше примере можно заметить несколько основных предложений (ключевых слов) SQL:

- **SELECT** — ключевое слово, которое используется для извлечения данных из БД;
- **FROM** — предложение, указывающее на то, из какого источника нужно извлечь данные.

Синтаксис

```
SELECT {выражение, то, что хотим вывести}  
FROM {таблица, из которой хотим вывести}
```

Используя после предложения **SELECT** знак звёздочки, мы выводим абсолютно все данные (строки и столбцы), которые есть в исследуемой таблице. В примере с таблицей **orders** получим следующий результат.

order_id	customer_id	merchant_id	order_amount	order_date	delivery_date	order_status	delivery_status
1	1	101	1000	2023-01-01	2023-01-03	paid	delivered
2	2	102	2000	2023-01-02	2023-01-04	paid	delivered
3	1	103	3000	2023-01-03	2023-01-06	completed	in transit
4	3	104	1500	2023-01-04	2023-01-05	completed	delivered
5	2	101	5000	2023-01-05	2023-01-07	paid	delivered

Очень часто в базах данных крупных компаний может встретиться десяток или даже сотня столбцов. В таком случае выводить абсолютно все данные не лучшая идея. Во-первых, с этим просто-напросто будет сложно работать. Во-вторых, в некоторых типах БД вывод всех столбцов может негативно сказаться на времени выполнения запроса, но об этом подробнее поговорим на одном из следующих занятий.

Как тогда поступить? Нужно указать после предложения **SELECT** список колонок, которые необходимы нам для анализа.

ПРИМЕР

Вывести только id заказа, сумму и дату совершения.

```
SELECT order_id  
        , order_amount  
        , order_date  
FROM orders
```

Тем самым мы создадим проекцию.

ОПРЕДЕЛЕНИЕ

Проекция — это операция, которая выбирает определённые столбцы из таблицы, создавая новый набор данных, содержащий только эти столбцы.

Также можно выводить столбцы, рассчитанные при помощи различных выражений.

ОПРЕДЕЛЕНИЕ

Выражения — это конструкции, состоящие из значений, операторов, функций и имён столбцов, которые вычисляются в определённое значение. Они используются для выполнения операций с данными в запросах.

ПРИМЕР

Вывести только id заказа, сумму, дату совершения и размер скидки, равный 5% от суммы.

```
SELECT order_id
       , order_amount
       , order_date
       , order_amount * 0.05
FROM orders
```

WHERE

Отлично, выводить определённые столбцы мы научились! Теперь стоит разобраться с тем, как ограничить число строк в результате запроса. Для этой цели существует предложение **WHERE** — в нём можно указать условие, по которому мы хотим провести фильтрацию исходной таблицы, создав тем самым выборку.

Синтаксис

```
SELECT {то, что хотим вывести}
FROM {таблица, из которой хотим вывести}
WHERE {условия}
```

ПРИМЕР

ПРОЧИТАТЬ всё ИЗ ТАБЛИЦЫ «заказы» ГДЕ «статус заказа» = «выполнен». В терминах языка SQL запрос будет выглядеть так.

```
SELECT *
FROM orders
WHERE order_status = 'completed'
```

Результат

order_id	customer_id	merchant_id	order_amount	order_date	delivery_date	order_status	delivery_status
3	1	103	3000	2023-01-03	2023-01-06	completed	in transit
4	3	104	1500	2023-01-04	2023-01-05	completed	delivered

При написании условий в **WHERE** можно использовать различные логические операторы.

- Сравнение: **>**, **<**, **>=**, **<=**, **=**.
- Также можно создать условие на неравенство исследуемого столбца определённому значению: **!=**, **<>**.
- Указать условие не на конкретное значение, а на список значений, используя операторы **IN** или **NOT IN**.
- Задавать условие на принадлежность интервалу при помощи **BETWEEN**.

Например, если бы мы хотели отобрать заказы с одним из следующих статусов: **completed**, **paid** — мы бы записали запрос следующим образом.

```
SELECT *  
FROM orders  
WHERE order_status IN ('completed', 'paid')
```

Кроме того, стоит отметить, что не все задачи могут быть решены с указанием единственного условия ограничения. Иногда может пригодиться составить выборку, которая будет одновременно удовлетворять нескольким условиям, — для такой задачи стоит в предложении **WHERE** объединить эти условия при помощи **AND** (логическое «и»).

```
SELECT *  
FROM orders  
WHERE условие1  
      AND условие2  
      AND условие3
```

Ну а если хотим отобрать данные, удовлетворяющие одному из условий, то вместо **AND** стоит использовать **OR** (логическое «или»).

```
SELECT *  
FROM orders  
WHERE условие1  
      OR условие2  
      OR условие3
```

Иногда для написания сложных условий приходится комбинировать несколько **OR** и **AND**. В данном случае для корректности выполнения запроса рекомендуется использовать скобки. Тут всё, как в математике: первым делом будут выполняться действия в скобках.

DISTINCT

Огоны! Мы уже познакомились с тремя предложениями SQL и теперь умеем не просто читать данные из базы, а делать это по определённым условиям. Однако как быть, если в выбранных данных оказались дубли (повторяющиеся строки)? Для борьбы с дублями в SQL существует несколько вариантов, но наиболее популярный из них — предложение **DISTINCT**.

Синтаксис

```
SELECT
  DISTINCT column_name
FROM table
```

Это предложение ставится в select-части нашего запроса, после него перечисляются колонки, уникальные комбинации которых мы хотим вывести. Например, перед нами стоит следующая задача.

ПРИМЕР

ПРОЧИТАТЬ все уникальные статусы ИЗ ТАБЛИЦЫ «заказы».
В терминах языка SQL запрос будет выглядеть так.

```
SELECT
  DISTINCT order_status
FROM orders
```

ORDER BY

Также популярной задачей является сортировка данных. Например, когда мы хотим не просто получить выгрузку с информацией о всех заказах нашей компании, а расположить заказы в ней от большего к меньшему. Для данной и других подобных задач в SQL существует предложение **ORDER BY**. Записывается **ORDER BY** после других основных предложений.

Синтаксис

```
SELECT {то, что хотим вывести}
FROM {таблица, из которой хотим вывести}
WHERE {условия}
ORDER BY {поля, по которым проводим сортировку}
```

ПРИМЕР

ПРОЧИТАТЬ всё ИЗ ТАБЛИЦЫ «заказы» ГДЕ «статус заказа» = «выполнен»
ОТСОРТИРОВАТЬ по убыванию.
В терминах языка SQL запрос будет выглядеть так.

```
SELECT *  
FROM orders  
WHERE order_status = 'completed'  
ORDER BY order_amount DESC
```

Виды сортировки

- **DESC** — сортировка по убыванию.
- **ASC** — сортировка по возрастанию.

Если не указать ни один из данных параметров, то по умолчанию будет использоваться сортировка по возрастанию (**ASC**).

Также важно отметить то, как проходит сортировка по разным типам данных.

- Если сортировка идёт по строковым типам данных, то она будет идти согласно алфавитному порядку и регистру.
- Сортировка по числам и датам интуитивно понятна. Если хотим отсортировать по возрастанию, то числа будут идти от меньшего к большему, а временные данные будут идти от более ранних к более поздним

А что, если нужно отсортировать по нескольким столбцам? Решая данную задачу, необходимо после **ORDER BY** указать все столбцы, по которым будем проводить сортировку. Также для каждого из столбцов можно указать тип сортировки (по возрастанию / по убыванию).

LIMIT

Работая с базами данных крупных компаний, можно увидеть, что в них хранятся миллионы строк данных. Выводить все строки — задача ресурсоёмкая. Но как тогда можно посмотреть на пример данных в таблице и потратить на это мало времени? Для этого существует оператор **LIMIT**, который записывается в самом конце запроса и показывает максимальное число строк, которые стоит вывести.

Синтаксис

```
SELECT {то, что хотим вывести}  
FROM {таблица, из которой хотим вывести}  
WHERE {условия}  
ORDER BY {поля, по которым проводим сортировку}  
LIMIT {число строк}
```

Например, запрос «Вывести 10 строк из таблицы orders» будет в SQL реализован следующим образом.

```
SELECT *  
FROM orders  
LIMIT 10
```

Использование условного оператора и встроенных функций SQL

Как можно было заметить, при работе с SQL приходится сталкиваться с различными типами данных, что уже не совсем просто. Сложнее становится, когда возникают задачи, требующие того или иного преобразования данных, написания тех или иных выражений, применение выражений в зависимости от условия. Благо для большинства потребностей в SQL уже реализованы специальные функции и операторы. Будем с ними знакомиться.

УСЛОВНЫЙ ОПЕРАТОР CASE

Иногда при создании колонки с помощью выражений возникает потребность в применении данного выражения в зависимости от того или иного условия. Например, если клиент девушка, скидка на цветы 10%, если мужчина, то 5%.

В SQL для этого существует оператор **CASE**.

Синтаксис

```
SELECT столбцы  
    , CASE  
        WHEN условие1 THEN значение1  
        WHEN условие2 THEN значение2  
        ELSE значение3  
FROM таблица
```


Также существует другой вариант реализации условной оператора **CASE**, который чаще всего используется для маппинга значений.

```
SELECT столбцы
, CASE выражение
    WHEN значение1 THEN результат1
    WHEN значение2 THEN результат2
    ELSE результат3
FROM таблица
```

СПЕЦИАЛЬНЫЕ ФУНКЦИИ ДЛЯ РАБОТЫ С ЧИСЛОВЫМИ ДАННЫМИ

Работать с числовыми типами данных достаточно просто, так как SQL поддерживает большинство математических операций (+, -, *, / и прочие).

```
SELECT 4*5 as multiplication
, 4+5 as addition
, 4-5 as subtraction
, 4/5 as division
```

Также существует ряд встроенных функций SQL, позволяющих выполнить те или иные математические операции.

Функция	Описание	Пример
POW(num, power)	Вычисляет число в указанной степени	SELECT POW(10, 2) -- вернёт 100
SQRT(num)	Вычисляет квадратный корень числа	SELECT SQRT(100) -- вернёт 10
LOG(base, num)	Вычисляет логарифм числа по указанному основанию	SELECT LOG(10, 100) -- вернёт 2
EXP(num)	Вычисляет e^{num}	SELECT EXP(2) -- вернёт 7.389
SIN(num)	Вычисляет синус числа (аналогично для косинуса, тангенса)	SELECT SIN(0) -- вернёт 0
ABS(num)	Вычисляет модуль числа	SELECT ABS(-100) -- вернёт 100

Следующая популярная задача связана с числовыми данными — это **округление**. Для неё в SQL также существует ряд встроенных функций.

Функция	Описание	Пример
CEIL(num)	Округляет число до целого в меньшую сторону	<code>SELECT CEIL(2.34) -- вернёт 2</code>
FLOOR(num)	Округляет число до целого в большую сторону	<code>SELECT FLOOR(10.2) -- вернёт 10</code>
ROUND(num, part)	Округляет число до указанного числа знаков после запятой. Без указания параметра part округляет до целого	<code>SELECT ROUND(10.2355, 2) -- вернёт 10.24</code>
TRUNCATE(num, part)	Обрезает число до указанного в параметре part знака после запятой	<code>SELECT TRUNCATE(10.2355, 2) -- вернёт 10.23</code>

В примерах выше мы использовали функции для конкретных чисел, но их также можно применять для столбцов в числовом типе данных. В таком случае выражение будет применяться отдельно к значению в каждой строке.

СПЕЦИАЛЬНЫЕ ФУНКЦИИ ДЛЯ РАБОТЫ С ТЕКСТОВЫМИ ДАННЫМИ

Когда мы работаем с колонкой, в которой находится текстовая информация, очень частыми задачами могут стать:

- определить всех клиентов, чьё имя начинается на букву А;
- определить всех клиентов, у кого электронная почта Google, и так далее.

Для решения подобных задач приходится заниматься поиском по шаблону.

LIKE

В SQL существуют операторы **LIKE** и **NOT LIKE**, которые записываются следующим образом.

```
строка LIKE шаблон [ESCAPE спецсимвол]
строка NOT LIKE шаблон [ESCAPE спецсимвол]
```

Выражение **LIKE** возвращает true, если строка соответствует заданному шаблону (**NOT LIKE**, если не соответствует). В шаблоне можно встретить несколько спецсимволов, например подчёркивание (**_**) в шаблоне подменяет (вместо него подходит) любой символ, а знак процента (**%**) подменяет любую (в том числе и пустую) последовательность символов.

Нижe представлено несколько примеров работы оператора **LIKE**.

```
'abc' LIKE 'abc'      true
'abc' LIKE 'a%'      true
'abc' LIKE '%a%'      true
'abc' LIKE '%a'       false
'abc' LIKE '_b_'      true
'abc' LIKE 'c'        false
```

Также похожую цель помогает достигнуть функция **SIMILAR TO** или использование знака `\~` (работает не во всех диалектах SQL).

Помимо поиска по шаблону, в SQL существует огромное количество функций, предназначенных для работы со строковым типом данных. Про всё многообразие можно почитать в соответствующей [документации](#), ниже рассмотрим наиболее популярные из функций.

Функция	Тип результата	Описание	Пример	Результат
string string	text	Конкатенация строк	'Post' 'greSQL'	PostgreSQL
char_length(string)	int	Число символов в строке	char_length('sql')	3
lower(string)	text	Переводит символы строки в нижний регист	lower('SQL')	sql
substring(string [from int] [for int])	text	Извлекает подстроку в заданном интервале	substring('Thomas' from 2 for 3)	hom
left(str text, n int)	text	Возвращает первые n символов в строке. Когда n меньше нуля, возвращаются все символы слева, кроме последних n	left('abcde', 2)	ab

СПЕЦИАЛЬНЫЕ ФУНКЦИИ ДЛЯ РАБОТЫ С ВРЕМЕНЕМ

Из всех типов данных в SQL временные данные являются наиболее сложными.

Трудности возникают по нескольким причинам, и вот некоторые из них:

- множество способов задания даты и времени;
- наличие временных зон;
- неочевидность вычислений некоторых значений на основании временных данных, например сложность вычисления возраста.

Разберём наиболее известные задачи, которые возникают при работе с временными данными.

Иногда необходимо получить не всю дату, а только её конкретную часть, например месяц или год.

Функции возвращающие часть даты

Функция	Описание
YEAR(datetime)	Возвращает год для указанной даты
MONTH(datetime)	Возвращает числовое значение месяца года (от 1 до 12) даты
DAY(datetime)	Возвращает порядковый номер дня в месяце (от 1 до 31)
HOURL(datetime)	Возвращает значение часа (от 0 до 23) для времени
MINUTE(datetime)	Возвращает значение минут (от 0 до 59) для времени

DATE_PART

Также существует функция **DATE_PART**, получающая на вход 2 параметра (первый — необходимая часть даты, второй — дата).

```
DATE_PART ( datepart , date )
```

EXTRACT

Ещё один вариант извлечь определённую часть из даты — функция **EXTRACT**. Основное отличие от предыдущего варианта в том, что **EXTRACT** возвращает результат типа double precision.

```
EXTRACT(field FROM source)
-- field: ключевое слово, определяющее, какую
часть source следует извлечь.
-- source: , DATE-, TIMESTAMP- или INTERVAL-выражение.
```

ПРИМЕР

Определить месяц у даты '2024-11-12 12:34:32' тремя способами.

```
SELECT MONTH('2024-11-12 12:34:32') as month_res
, DATE_PART('month', '2024-11-12 12:34:32')
as datepart_res
, EXTRACT(MONTH FROM '2024-11-12 12:34:32')
as extract_res
```

Класс, мы научились вытаскивать определённую часть даты. Теперь предположим, что нам нужно найти всех клиентов, которые совершали заказ в нашем интернет-магазине в марте 2024 года.

Ниже приведём два решения этой задачи, а после разберём их недостатки.

```
-- 1
SELECT *
FROM orders
WHERE DATE_PART('month', order_date) = 3

-- 2
SELECT *
FROM orders
WHERE DATE_PART('month', order_date) = 3
      AND DATE_PART('year', order_date) = 2024
```

Очевидно, что первый вариант выведет нам все заказы в марте, однако если предположить, что наш интернет-магазин функционирует уже более 5 лет, то запрос выведет нам информацию о заказах не только за 2024 год.

Ок, добавим условие на год, как это сделано во втором примере. Действительно, такой запрос выведет уже корректную информацию, но нам уже нужно прописывать несколько условий, что не всегда удобно и занимает больше времени.

Существует ли функция, которая поможет решить задачу одним условием?

Да, функция называется **DATE_TRUNC**.

На вход она получает два параметра:

- единицу времени, до которой округляем дату;
- дату, которую нужно округлить.

По сути, данная функция будет приводить дату к началу периода по выбранной единице времени, пример ниже.

DATE_TRUNC	Результат
date_trunc('month', '2024-03-14')	'2024-03-01'
date_trunc('year', '2024-03-14')	'2024-01-01'
date_trunc('week', '2024-03-14')	'2024-03-10'

Вычисления с датами

Следующая частотная задача — провести вычисления с датой, например, найти возраст человека или учитывать данные только за последние 2 недели. Чтобы решить данную задачу, в SQL для временных данных поддерживаются математические операторы (+, −, *), а также существуют специальные функции.

Оператор	Пример	Результат
+	date '2001-09-28' + integer '7'	date '2001-10-05'
+	date '2001-09-28' + interval '1 hour'	timestamp '2001-09-28 01:00:00'
+	date '2001-09-28' + time '03:00'	timestamp '2001-09-28 03:00:00'
+	time '01:00' + interval '3 hours'	time '04:00:00'
−	date '2001-10-01' - date '2001-09-28'	integer '3' (дня)
−	date '2001-10-01' - integer '7'	date '2001-09-24'
−	date '2001-09-28' - interval '1 hour'	timestamp '2001-09-27 23:00:00'
−	timestamp '2001-09-29 03:00' - timestamp '2001-09-27 12:00'	interval '1 day 15:00:00'
*	900 * interval '1 second'	interval '00:15:00'
*	21 * interval '1 day'	interval '21 days'
*	double precision '3.5' * interval '1 hour'	interval '03:30:00'
DATEDIFF (datepart , startdate , enddate)	DATEDIFF('month', '2017/08/25', '2011/08/25')	-72

Важно:



- при помощи математических операторов можно прибавлять/вычитать различные типы данных (integer, interval, time);
- функция **DATEDIFF** есть не во всех диалектах SQL, её аналогом может послужить функция **EXTRACT**, у которой после **FROM** указывается разница двух дат. Для наибольшей точности можно использовать **EXTRACT(epoch FROM date1 - date2)**, что вернёт число секунд между датами.

Текущее время

Ещё очень часто возникает потребность составить условие относительно текущего момента времени или, например, написать универсальный скрипт, который будет выгружать данные о клиентах, которые совершали заказ за последние 2 недели. В SQL для этого существует несколько функций, с которыми точно нужно быть знакомым:

- функция **CURRENT_DATE** возвращает текущую дату;
- функция **NOW()** возвращает текущий момент времени в формате «год-месяц-день часы:минуты:секунды».

Специальные функции для работы с пропусками

Иногда значение в одной из строк того или иного столбца может быть ничем не заполнено.

В данном случае в SQL на этом месте будет стоять пропуск (**NULL**). Какие у него есть особенности?

- **NULL** не эквивалентно пустой строке (для текстовых полей) или нулю (для числовых полей).
- Если один из операндов в арифметическом выражении равен **NULL**, результат всего выражения также будет **NULL**.
- Сравнение **NULL** с любым значением, включая другой **NULL**, возвращает **UNKNOWN**, а не **TRUE** или **FALSE**. Поэтому мы не можем при работе с **NULL** использовать обычные операторы сравнений. Для проверки на **NULL** используются операторы **IS NULL** и **IS NOT NULL**.

Также существует несколько специальных функций по работе с **NULL**-значениями.

- **COALESCE(значение1, значение2, ...)**:
возвращает первое не **NULL**-значение из списка.
- **NULLIF(значение1, значение2)**:
если значения равны, возвращает **NULL**, иначе — значение1.
- **IFNULL(значение, замещающее_значение)**:
Если значение не **NULL**, возвращает его, иначе — замещающее_значение.

ПАМЯТКА С ИЗУЧЕННЫМИ ПРЕДЛОЖЕНИЯМИ

Сегодня мы познакомились с большим количеством материалов. Чтобы в голове после прочтения не осталось каши, соберём единую памятку по основным предложениям SQL.

Предложение	Что необходимо понять?
SELECT	Синтаксис SELECT {указываем то, что нужно вывести} • Отвечает за то, какие данные будут выведены по итогу запроса. • После него нужно указать названия столбцов, которые нужно вывести. • Если нужно вывести всю информацию, то проще указать SELECT *. • Чтобы вывести только уникальные значения, после SELECT стоит указать DISTINCT
FROM	Синтаксис SELECT {указываем то, что нужно вывести} FROM {указываем, из какой таблицы нужно выводить данные} • Отвечает за то, из какой базы данных будет производиться вывод данных, указанных в SELECT • Записывает после завершённого блока SELECT
WHERE	Синтаксис SELECT {указываем то, что нужно вывести} FROM {указываем, из какой таблицы нужно выводить данные} WHERE {условие} AND/OR {условие} ... • Условный оператор, при помощи которого фильтруются данные в результирующем датасете. • Можно указывать несколько одновременных условий, используя между условиями AND • Используя OR, можем оставлять данные, удовлетворяющие одному из условий. • Если в условиях существует определённый приоритет, то стоит использовать скобки для его обозначения. Тут всё как в математике: сначала проверяются условия в скобках, далее проверяются условия вне скобок
ORDER BY	Синтаксис SELECT {указываем то, что нужно вывести} FROM {указываем, из какой таблицы нужно выводить данные} WHERE {условие} AND/OR {условие} ... ORDER BY {указываем поля сортировки} • Сортирует результирующий датасет по указанным после оператора ORDER BY столбцам. • Для сортировки по убыванию указываем DESC, для сортировки по возрастанию — ASC. • Можно указывать разный порядок сортировки для разных столбцов. Например, сортировать по убыванию цены, а если цена одинаковая, то сортировать по возрастанию оценки
LIMIT	Синтаксис SELECT {указываем то, что нужно вывести} FROM {указываем, из какой таблицы нужно выводить данные} WHERE {условие} AND/OR {условие} ... ORDER BY {указываем поля сортировки} LIMIT {указываем число строк, которое нужно вывести} Указывает максимальное число строк результирующего датасета, которое будет выведено в консоль

Заключение

В этом материале мы разобрали самые основные операторы SQL, позволяющие выполнять простые запросы к базам данных, читать данные из БД, добавлять к ним условия, а также сортировать результирующий датасет. Познакомились с наиболее часто используемыми типами данных, разобрали функции, специфичные для определённого набора типов данных. Теперь ты можешь спокойно изучить данные, которые хранятся в интересующей тебя таблице, и доработать свои условия при помощи специальных функций, если исходных признаков недостаточно.